MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 2
ALGORİTMA KAVRAMI VE ANALİZİ

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 2

- Algoritma
- Algoritma Süreci
- Algoritmaların Gösterimi
- Algoritmanın Etkinliği
- Algoritma Analizi
- Yürütme Zamanı Analizi

Algoritma

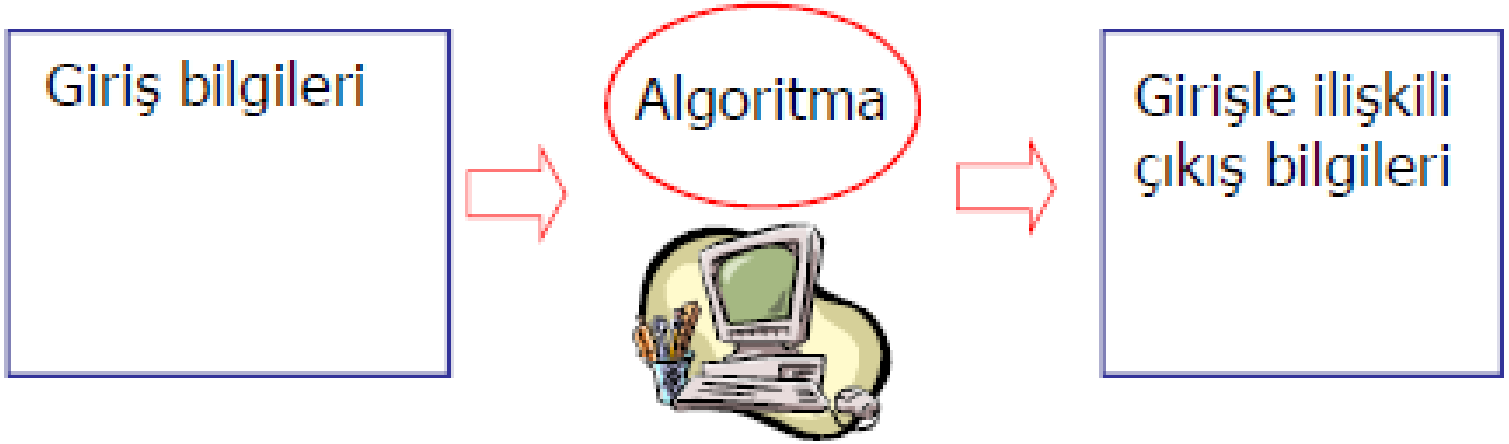
Genel Bakış

- Algoritma, 800'lü yıllarda yaşamış olan İranlı matematikçi Muhammad ibn Musa al-Khwärizmi'nin yaptığı çalışmalarda ortaya konmuştur.
- İlk algoritma, el-Hârizmî tarafından "Hisab el-cebir ve el-mukabala" kitabında sunulmuştur.
- 12. yüzyılda bu çalışma Latince'ye çevrilirken, çalışmaların sahibi olan el-Hârizmî'nin adından dolayı "algorithm" olarak çevrilmiştir.
- Bu kelime Türkçe'ye ise algoritma olarak girmiştir.
- Tarihçesinden de anlaşılacağı üzere algoritma, matematik alanındaki problemlerin çözümü için kullanılmaktaydı.
- Daha sonra bilgisayarların geliştirilmesiyle bu alandaki problemlerin çözümünde de kullanılmaya başlandı.

Algoritma

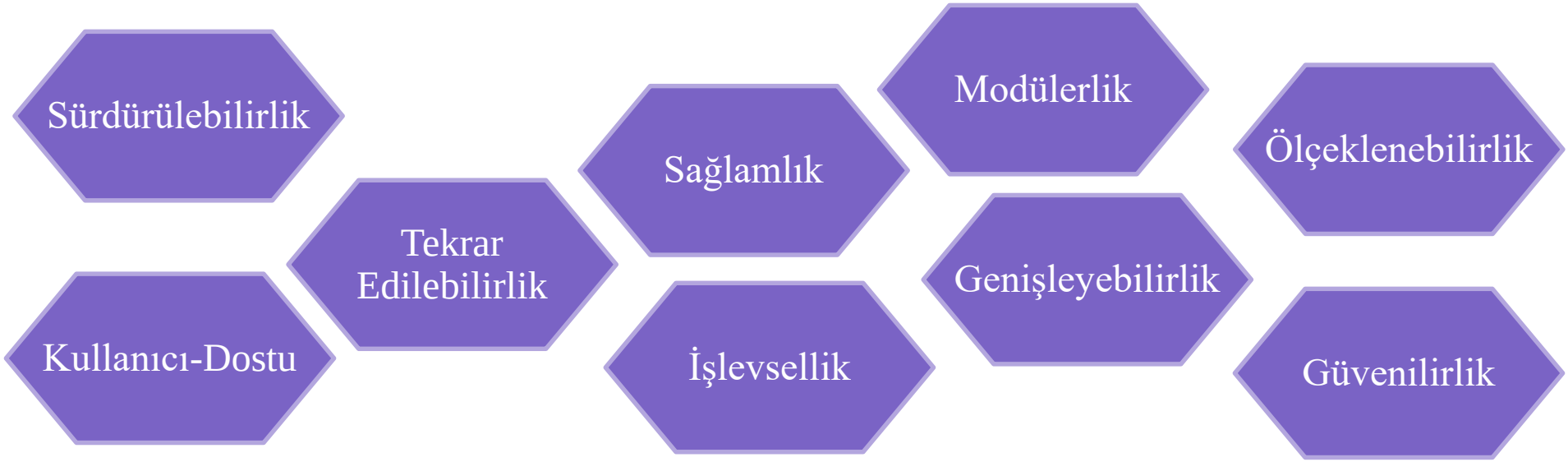
➢ Tanım

- Bir problemi çözmek için sonlu sayıdaki adım adım birbirini takip eden işlemler kümesidir.
- Algoritma, bir programlama dili ile ifade edildiğinde program adını alır.
- Belirli bir problemin sonucu için uygulanacak adımları ve koşulları kesin olarak ortaya koyar.

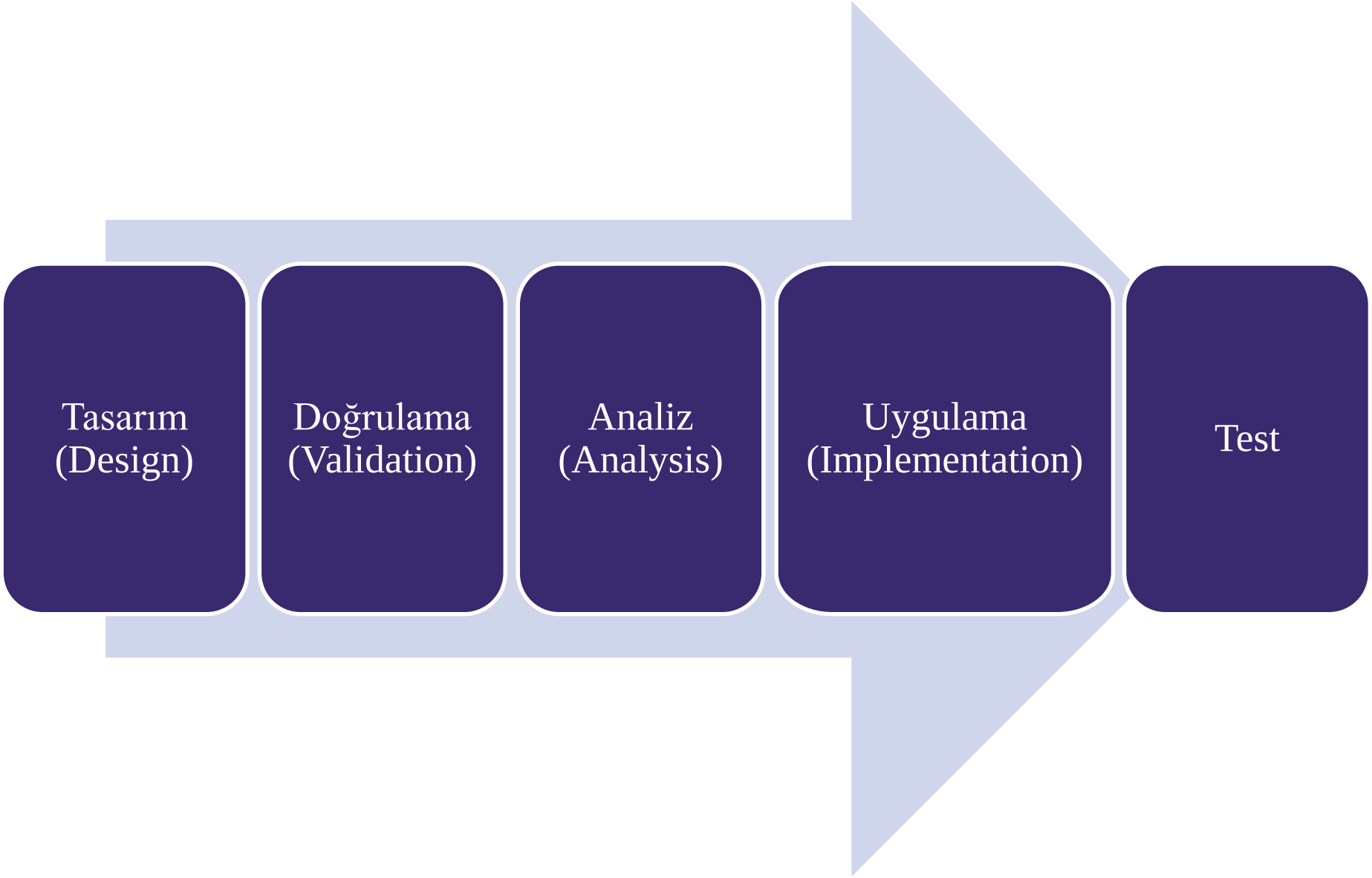


Algoritma

- Özellikleri
  1. Girişi/Çıkışı olmalıdır
  2. Doğru olmalıdır
  3. Her adımı tanımlı olmalıdır
  4. Sonlu sayıda adıma sahiptir
  5. Etkin olmalıdır (zamansal/uzaysal analiz)



Algoritma Süreci



Algoritmaların Gösterimi

- Algoritmayı belirtmek için
  - Kaba kod (Pseudo code)
  - Akış diyagramı
  - N-S (Nassi-Schnederman) Şeması
  - W-O (Warnier-Orr) Diyagramı

olmak üzere 4 yöntem kullanılır.

Algoritmaların Gösterimi

- Kaba Kod (Pseudo Code)
  - Bir programlama dilinden ziyade programcının dilinde yazılmış ön hazırlık kodudur.
  - Gerçek bir programlama dilinin sözdizimi kurallarını taşımaz fakat genel anlaşılabilecek programlama deyimlerini içerebilir.
  - Sadece genel olarak algoritmanın işleyişini gösterir.

Algoritmaların Gösterimi

- Kaba Kod (Pseudo Code)
  - Örnek: Kullanıcının girdiği dört sayının ortalamasını görüntüleyen algoritma:

    A0 --> Başla

    A1 --> Sayaç=0 (Sayaç'ın ilk sayısı 0 olarak başlar.)

    A2 --> Sayı=? : T=T+Sayı (Sayıyı giriniz. T'ye sayıyı ekle ve T'yi göster.)

    A3 --> Sayaç=Sayaç+1 (Sayaç'a 1 ekle ve sayacı göster.)

    A4 --> Sayaç<4 ise A2'ye git. (Eğer sayaç 4'ten küçükse Adım 2'ye git.)

    A5 --> O=T/4 (Ortalama için T değerini 4'e böl)

    A6 --> O'yu göster. (Ortalamayı göster.)

    A7 --> Dur

Algoritmaların Gösterimi

➢ Kaba Kod (Pseudo Code) – Temel Gösterim

- Bir değer atamak için genellikle = kullanılır. == işareti ise eşitlik kontrolü için kullanılır.
- Metot, fonksiyon, yordam isimleri:
  - Metot Adı ({parametre listesi})
- Program yapısı şu şekilde tanımlanır:
  - Karar yapıları: if ... then ... else ...
  - while döngüleri: while ... do {döngü gövdesi}
  - for döngüleri: for ... {döngü gövdesi}
  - Dizi indeksleri: A[i]
- Metotların çağrılması:
  - Metot Adı ({değişken listesi})
- Metotlardan geri dönüş: return değer

Algoritmaların Gösterimi

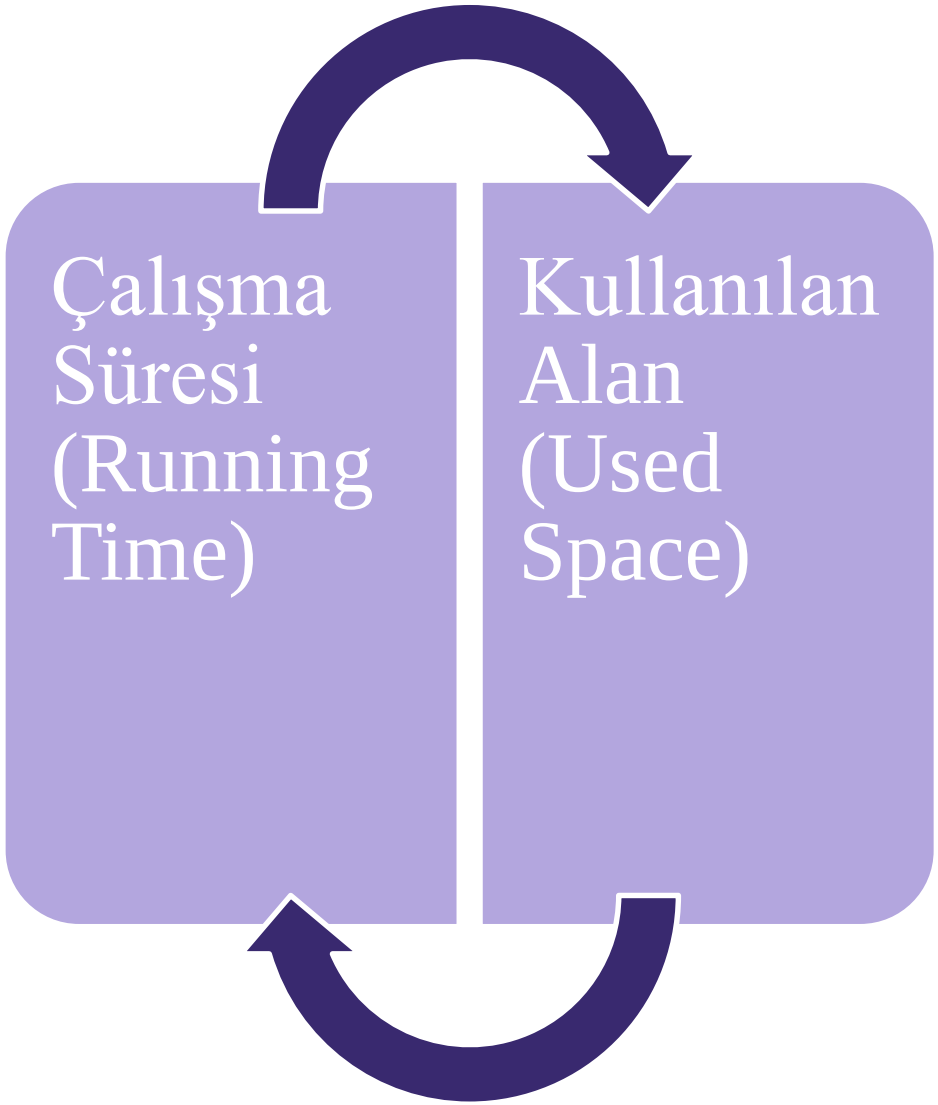
➢ Kaba Kod (Pseudo Code) – Temel Gösterim

- Örnek: Bir dizinin en küçük elemanını bulan algoritma:

```
en küçük = Dizinin ilk elemanı
for (ikinci elemandan dizi sonuna  kadar)
        if (sıradaki eleman < en küçük)
                en küçük = sıradaki eleman
en küçük elemanı yazdır veya  gönder
```

```
1  bulenkucuk(int a[])
2  {
3  int enkucuk = a[0];
4      for (int k=1; k<a.Length; k++)
5          if(a[k]<enkucuk)
6              enkucuk=a[k];
7  return enkucuk;
8  }
```

Algoritmanın Etkinliği



Algoritmanın Etkinliği

- Bir algoritmanın etkinliği, uygulama öncesi ve sonrasında iki farklı aşamada analiz edilebilir.
- Öncelikli (Priori) Analiz:
  - Bu bir algoritmanın teorik bir analizidir. Bir algoritmanın verimliliği, diğer tüm faktörlerin, örneğin işlemci hızının sabit olduğu varsayılarak ölçülür. Bunda, algoritmanın verimliliği, karmaşıklığına göre hesaplanır. Bu sadece yaklaşık bir analizdir.
- Kaba (Posterior) Analiz:
  - Bu, bir algoritmanın ampirik bir analizidir. Seçilen algoritma, programlama dili kullanılarak gerçekleştirilir. Bu daha sonra hedef bilgisayar makinesinde yürütülür. Bu analizde, çalışma süresileri toplanıp değerlendirilir. (C için <time.h> kütüphanesi kullanılabilir.)

Algoritma Analizi

- Algoritma analizi, tasarlanan program veya fonksiyonun belirli bir işleme göre matematiksel ifadesini bulmaya dayanır.
- Burada temel hesap birimi seçilir ve programın görevini yerine getirebilmesi için bu işlemden kaç adet yapılması gerektiğini bulmaya yarayan bir bağıntı hesaplanır.
- Eğer bu bağıntı zamanla ilgiliyse çalışma hızını, bellek gereksinimiyle ilgiliyse bellek gereksinimi ortaya koyar.

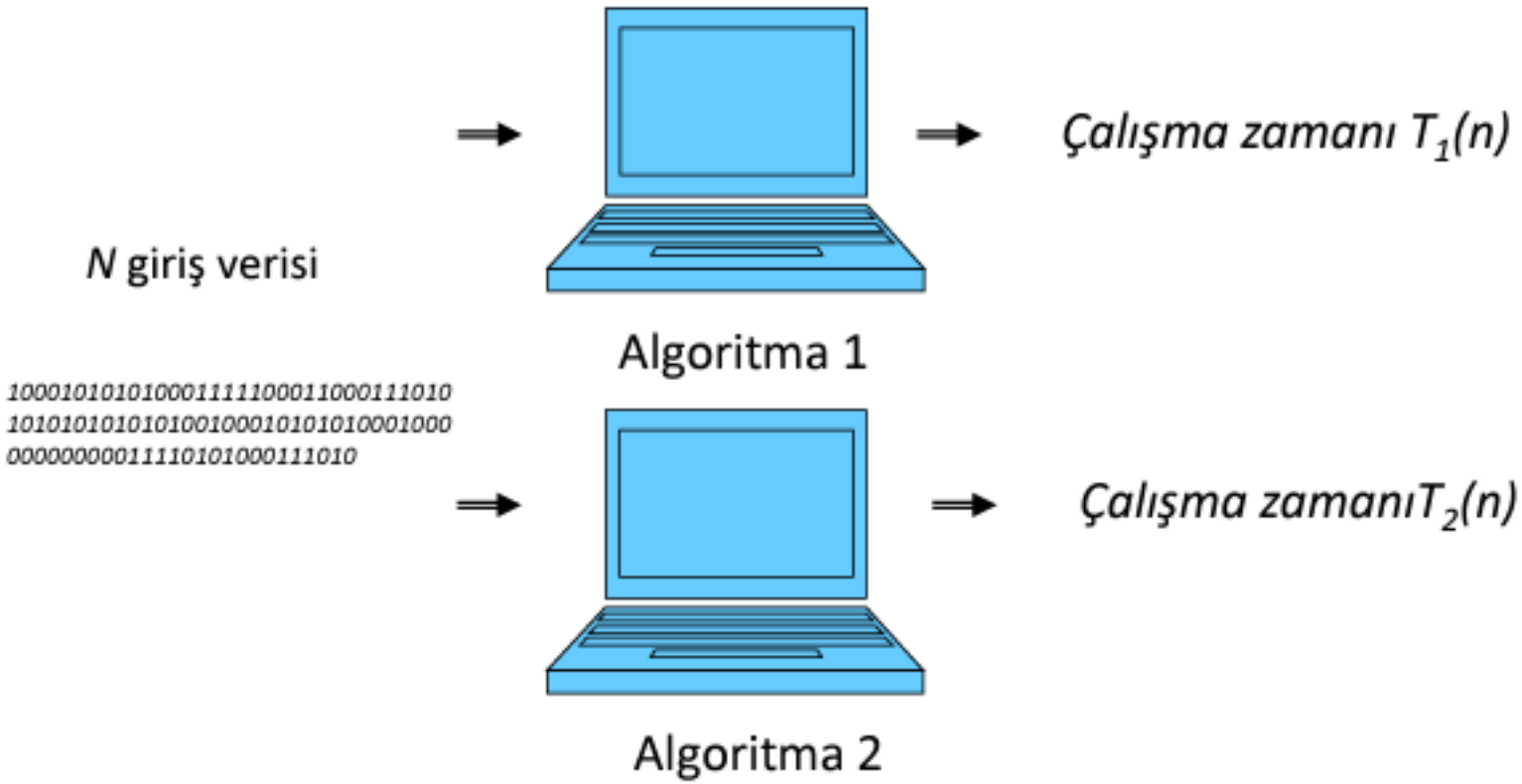
Algoritma Analizi

- Neden algoritmayı analiz ederiz?
  - Algoritmanın performansını ölçmek için
  - Farklı algoritmalarla karşılaştırmak için
  - Daha iyisi mümkün mü? Olabileceklerin en iyisi mi?
- Algoritmayı nasıl analiz ederiz?
  - Yürütme zamanı (Running Time) - T(n)
  - Karmaşıklık (Complexity) - Notasyonlar

Yürütme Zamanı Analizi

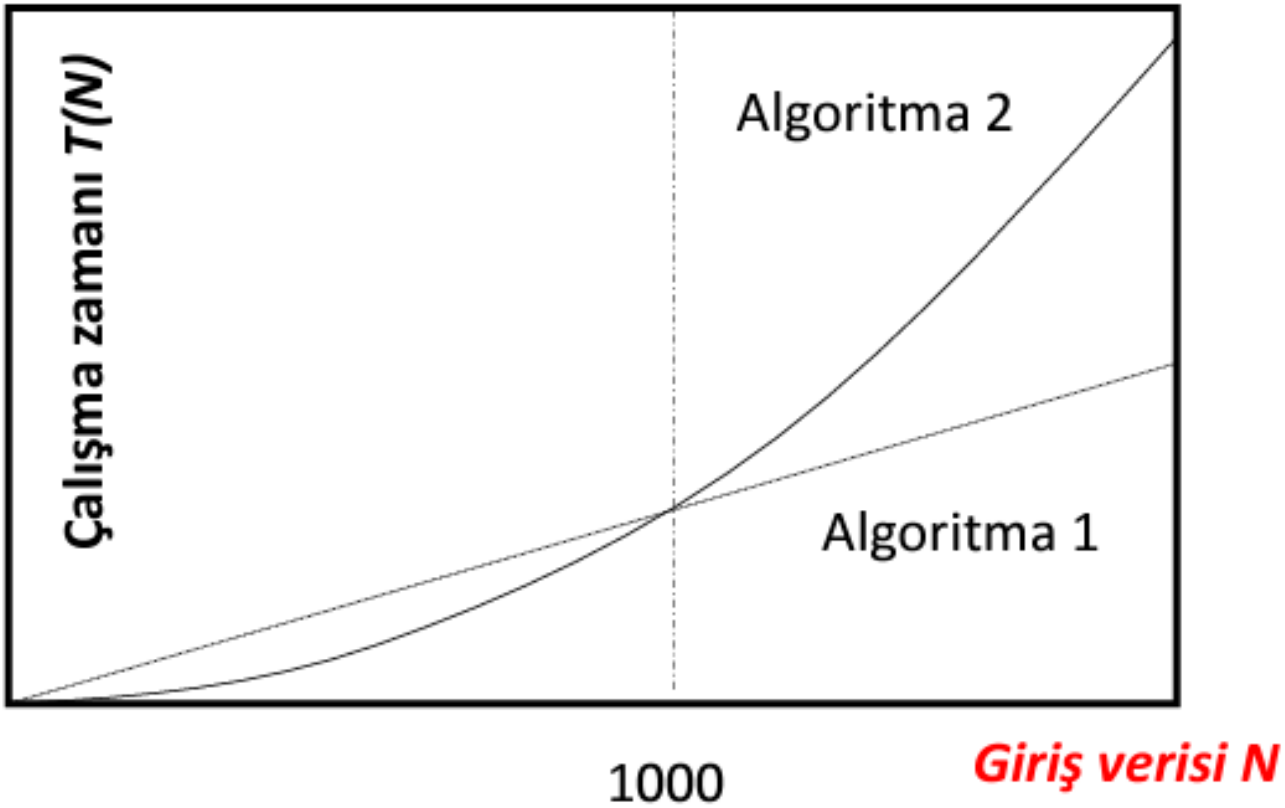
- Running Time Analysis - T(n)
- Yürütme Zamanı; Bir programın veya algoritmanın işlevini yerine getirebilmesi için, temel kabul edilen işlevlerden kaç adet yürütülmesini veren bir bağıntıdır ve T(n) ile gösterilir.
- Temel hesap birimi olarak, programlama dilindeki deyimler seçilebildiği gibi döngü sayısı, toplama işlemi sayısı, atama sayısı, dosyaya erişme sayısı gibi işler de temel hesap birimi olarak seçilebilir.

Yürütme Zamanı Analizi



- Algoritma 1, T1(N)=1000N
- Algoritma 2, T2(N)=N^2

Yürütme Zamanı Analizi



- Algoritma 1, T1(N)=1000N
- Algoritma 2, T2(N)=N^2

Yürütme Zamanı Analizi

N	T1	T2
10	10^{-2} sec	10^{-4} sec
100	10^{-1} sec	10^{-2} sec
1000	1 sec	1 sec
10000	10 sec	100 sec
100000	100 sec	10000 sec

- N değerinin 1000'den küçük olduğu durumlarda iki algoritma arasındaki çalışma zamanı ihmal edilebilir büyüklüktedir.

Yürütme Zamanı Analizi

- Örnek: Aşağıda bir dizinin aritmetik ortalamasını bulan ve sonucu çağırana gönderen bulOrta() fonksiyonun kodu verilmiştir. Bu fonksiyonun yürütme zamanını gösteren T(n) bağıntısını belirleyelim.

```
float bulOrta (float A[], int n) {
{
        float ortalama, toplam=0;
        int k ;
        for(k=0;k<n;k++)
                toplam+=A[k];
        ortalama=toplam/n;
        return ortalama;
}
```

Yürütme Zamanı Analizi

- Çözüm:

	Birim Zaman	Tekrar Sayısı	Toplam
float bulOrta (float A[], int n) {	-	-	-
float ortalama, toplam=0;	-	-	-
int k ;	-	-	-
for(k=0;k<n;k++)	1,1,1	1, (n+1), n	2n+2
toplam+=A[k];	1	n	n
ortalama=toplam/n;	1	1	1
return ortalama;	1	1	1
}	-	-	-
			T(n)=3n+4

Yürütme Zamanı Analizi

- Örnek: Aşağıda bir dizi içerisindeki en küçük elamanı bulan bulEnKucuk() adlı bir fonksiyonun kodu görülmektedir. Bu fonksiyonun yürütme zamanını gösteren T(n) bağıntısını belirleyelim.

```
float bulEnKucuk(float  A[])    {
    float enkucuk;
    int k ;
    enkucuk=A[0];
    for(k=1;k<n;k++)
        if (A[k]<enkucuk)
            enkucuk=A[k];
    return enkucuk;
    }
```

Yürütme Zamanı Analizi

- Çözüm:

	Birim Zaman	Tekrar Sayısı	Toplam
float bulEnKucuk(float A[]) {	-	-	-
float enkucuk;	-	-	-
int k ;	-	-	-
enkucuk=A[0];	1	1	1
for(k=1;k<n;k++)	1,1,1	1, n, (n-1)	2n
if (A[k]<enkucuk)	1	n-1	n-1
enkucuk=A[k];	1	n-1	n-1
return enkucuk;	1	1	1
}	-	-	-
			T(n)=4n

Yürütme Zamanı Analizi

- Örnek: İki matrisin toplanması için T(n) hesabını yapalım

	Birim Zaman	Tekrar Sayısı	Toplam
void toplaMatris (A,B,C) {	-	-	-
int A[n][m], B[n][m], C[n][m];	-	-	-
int i,j ;	-	-	-
for(i=0;i<n;i++)	1,1,1	1,(n+1),n	2n+2
for(j=0;j<m;j++)	1,1,1	n(1,(m+1),m)	n(2m+2)
C[i][j]=A[i][j]+B[i][j];	1	nm	nm
}	-	-	-
			T(n,m)=3nm+4n+2

Yürütme Zamanı Analizi

- Çıkarımlar
  - Çalışma zamanının kesin olarak belirlenmesi zordur
  - Giriş verilerine bağlı olan
    - En iyi durum (best case)
    - Ortalama durum (average case) hesaplanması zordur
  - Diğerlerine göre en kötü durum (worst case) hesaplanması kolaydır.
  - Bunun için çeşitli notasyonlardan faydalanılır.